

Rapport de pentest — Hermanas

Branche master (43 commits + modifications non commitées)

Revue sécurité automatisée

30 mai 2026

Synthèse

Aucune vulnérabilité haute ou moyenne avec confiance $\geq 0,7$ n'a été identifiée sur les changements introduits par cette branche.

La nouvelle surface d'attaque a été examinée méthodiquement :

- CRUD Résidents avec upload de photo
 - Sous-système Push Notifications
 - Nouveaux endpoints Config REST
 - Modifications de SecurityConfig
 - Changements frontend (Residents + monthly-trend-chart)
-

Zones revues — pourquoi elles passent

ResidentRestController + ResidentPhotoStorage

Fichiers : `src/main/java/org/jibe77/hermanas/web/`

`ResidentRestController.java`, `src/main/java/org/jibe77/hermanas/service/resident/ResidentPhotoStorage.java`

- Noms de fichiers générés serveur via `UUID.randomUUID()` + extension — jamais contrôlés par le client.
- `resolve()` applique à la fois `.normalize()` et un check `startsWith(photosPath)` → **path traversal bloqué**.
- L'extension du fichier est dérivée de la whitelist MIME validée (`image/jpeg|png|webp`) ; le `Content-Type` servi au download correspond à l'extension → un polyglot HTML déguisé en JPEG ne peut pas s'exécuter dans le navigateur.
- Plafond 5 MB et whitelist appliqués avant écriture.
- Auth : `POST/PUT/DELETE` couverts par la règle `URL hasAnyRole(USER, ADMIN)` ; `GET` (listing + photo) volontairement public selon le design “dashboard public en lecture seule”.

SecurityConfig

Fichier : `src/main/java/org/jibe77/hermanas/security/`

`SecurityConfig.java`

- CSRF activé via `CookieCsrfTokenRepository.withHttpOnlyFalse()` (cookie + header `X-XSRF-TOKEN`).
- Le correctif `hasRole(USER)` → `hasAnyRole(USER, ADMIN)` est bien en place.
- Nouvelles règles `/api/v1/push/**` correctes : VAPID public, `/test` admin-only, `subscribe/unsubscribe` couvert par la règle `POST` générique.
- Restriction `/api/v1/camera/**` → `/api/v1/camera/photos/**` cohérente avec l'intention.

PushRestController + PushNotificationService

- `broadcast()` n'est appelé qu'avec des chaînes hardcodées (DoorEventService + /test admin) → l'échappement JSON partiel n'est pas atteignable par de l'input utilisateur.
- L'endpoint côté abonné (passé à la lib Web Push) déclenche un POST aveugle signé VAPID, sans credentials utilisateur ni reflet de la réponse → **ne satisfait pas le critère SSRF reportable**.

ConfigRestController

- Clé API météo correctement **masquée** dans GET /api/v1/config (uniquement `key_set` + `key_length` exposés).
- Tous les setters gated par `@PreAuthorize("hasRole('USER')")` + règle URL.

RegistrationService + UserRestController

- L'inscription self-service assigne toujours `NOT_VALIDATED_YET` → pas d'injection de rôle.
- `updateMe()` met explicitement `body.setRole(null)` avant application → **pas d'auto-élévation**.
- CRUD admin gated par `hasRole('ADMIN')` + protection "last admin" sur `demote/delete`.

Frontend (Residents + monthly-trend-chart)

- grep sur `bypassSecurityTrust, innerHTML, [innerHTML]` dans `frontend/src/**` → **zéro occurrence** sur les nouveaux fichiers.
 - `photoSrc(r)` consomme un `photoUrl` généré côté serveur (ResidentMapper hardcode `/api/v1/residents/{id}/photo`), pas de l'input utilisateur.
 - Interpolation Angular auto-échappée partout.
-

Observations sous le seuil (non formelles)

À titre informatif uniquement, pour le mainteneur :

1. **PushNotificationService.escape()** — `src/main/java/org/jibe77/hermanas/service/push/PushNotificationService.java:195`
Échappement JSON partiel (`\`, `"`, `\n` seulement). Les appelants actuels passent uniquement des chaînes hardcodées, mais si un titre/corps user-controlled y est branché plus tard, les contrôles `\r\t\b\f` et `< 0x20` passeraient au travers. *Recommandation* : préférer `ObjectMapper.writeValueAsString` si ce cas se présente.
 2. **PushRestController** `subscribe/unsubscribe` ne vérifie pas que l'utilisateur appelant possède l'endpoint qu'il modifie. Exploitation conditionnée à deviner l'URL push d'un autre user (URL capability longue et aléatoire) → impact pratique faible.
 3. **ConfigRestController** annotation classe
`@PreAuthorize("hasRole('USER')")` : les admins purement ADMIN (sans USER) reçoivent 403 → **bug fonctionnel, pas d'exposition**.
-

Conclusion

Les nouveaux endpoints introduisent une surface d'attaque maîtrisée :

- upload photo bordé (whitelist MIME + UUID + path-traversal guard) ;
- CRUD Résidents avec authz alignée sur le modèle existant ;
- endpoints Config qui ne fuient pas la clé API.

Les évolutions de SecurityConfig conservent CSRF activé et un modèle de rôles cohérent.

Verdict : RAS exploitable sur cette branche. Les trois observations sous-seuil ci-dessus relèvent du durcissement défense-en-profondeur.